

Exam-Scheduler

Student: Toth Antonio Roberto

Grupa: 30238

1. Introducere

Aplicația "Exam-Scheduler" este o soluție web dezvoltată pentru a automatiza și eficientiza procesul de planificare a examenelor în cadrul unei instituții educaționale. Scopul principal este eliminarea conflictelor de orar, gestionarea eficientă a capacităților sălilor și oferirea unui portal dedicat și securizat pentru studenți, profesori și administratori. Aplicația va asigura o experiență fluidă, separând logica complexă de programare prin intermediul unei arhitecturi bazate pe microservicii.

2. Tehnologii

- Backend - Central Service: Java (cu Spring Boot)
- Backend - Scheduling Microservice: Node.js (cu Express) sau Python (cu FastAPI) : Utilizate pentru performanța ridicată în procesarea algoritmilor de căutare și validare a conflictelor, permițând o execuție rapidă a logicii de business complexe.
- Frontend: React.js sau Angular.
- Database: PostgreSQL

3. Arhitectura software

Aplicația folosește o arhitectura n-layer, bazată pe microservicii.

- Frontend Layer (Client): Responsabil pentru interfața cu utilizatorul. Comunica cu backend-ul prin apeluri API (REST) și utilizează "Routing Guards" pentru a bloca accesul neautorizat la anumite pagini.
- API Gateway / Central Service (Backend Layer 1): Acționează ca punct unic de intrare. Gestionează autentificarea, autorizarea (rolurile) și rutează cererile mai departe.
- Scheduling Service (Backend Layer 2): Un microserviciu separat dedicat logicii de business complexe (calcularea disponibilității sălilor, verificarea conflictelor de orar).
- Data Access Layer (Database): Fiecare microserviciu își poate gestiona propriile date sau pot accesa o bază de date centrală în mod securizat.

4. Functional Requirements

- Sistemul trebuie sa permită autentificarea utilizatorilor pe baza de adresa de email si parola, alocând-le roluri specifice (Student, Profesor, Admin).
- Profesorii trebuie sa poată solicita adăugarea unui examen, selectând cursul, durata si tipul acestuia.
- Sistemul trebuie sa valideze solicitările si sa respingă automat creările de examene care se suprapun în aceeași sala la aceeași ora.
- Studenții trebuie sa aibă acces la un dashboard unde sa vizualizeze doar orarul examenelor pentru cursurile la care sunt înrolați.
- Managementul Conflictelor de Timp: Sistemul trebuie să asigure o perioadă de repaus (buffer) de minimum 15 minute între două examene programate în aceeași sală pentru a permite igienizarea și pregătirea spațiului.
- Sistem de Notificări: La confirmarea unei programări de către microserviciul de Scheduling, sistemul trebuie să trimită automat un e-mail de confirmare profesorului și să actualizeze dashboard-ul studenților înrolați în timp real.
- Filtrare și Căutare Avansată: Studenții trebuie să poată filtra examenele după data calendaristică, tipul examenului (scris/oral) sau curs, pentru o mai bună organizare a timpului de studiu.

5. Non-Functional Requirements

- Securitate: Parolele utilizatorilor vor fi stocate folosind hashing (ex. BCrypt). Accesul la rutele Frontend-ului si la endpoint-urile Backend-ului se va face pe baza de token-uri JWT.
- Performanta: Generarea si încărcarea orarului in panoul studentului trebuie sa dureze mai puțin de 2 secunde in condiții normale de rețea.
- Scalabilitate: Datorita folosirii microserviciilor, componenta de "Scheduling" trebuie sa poată fi scalata independent in perioadele de sesiune, când traficul este maxim.

6. Diagrams

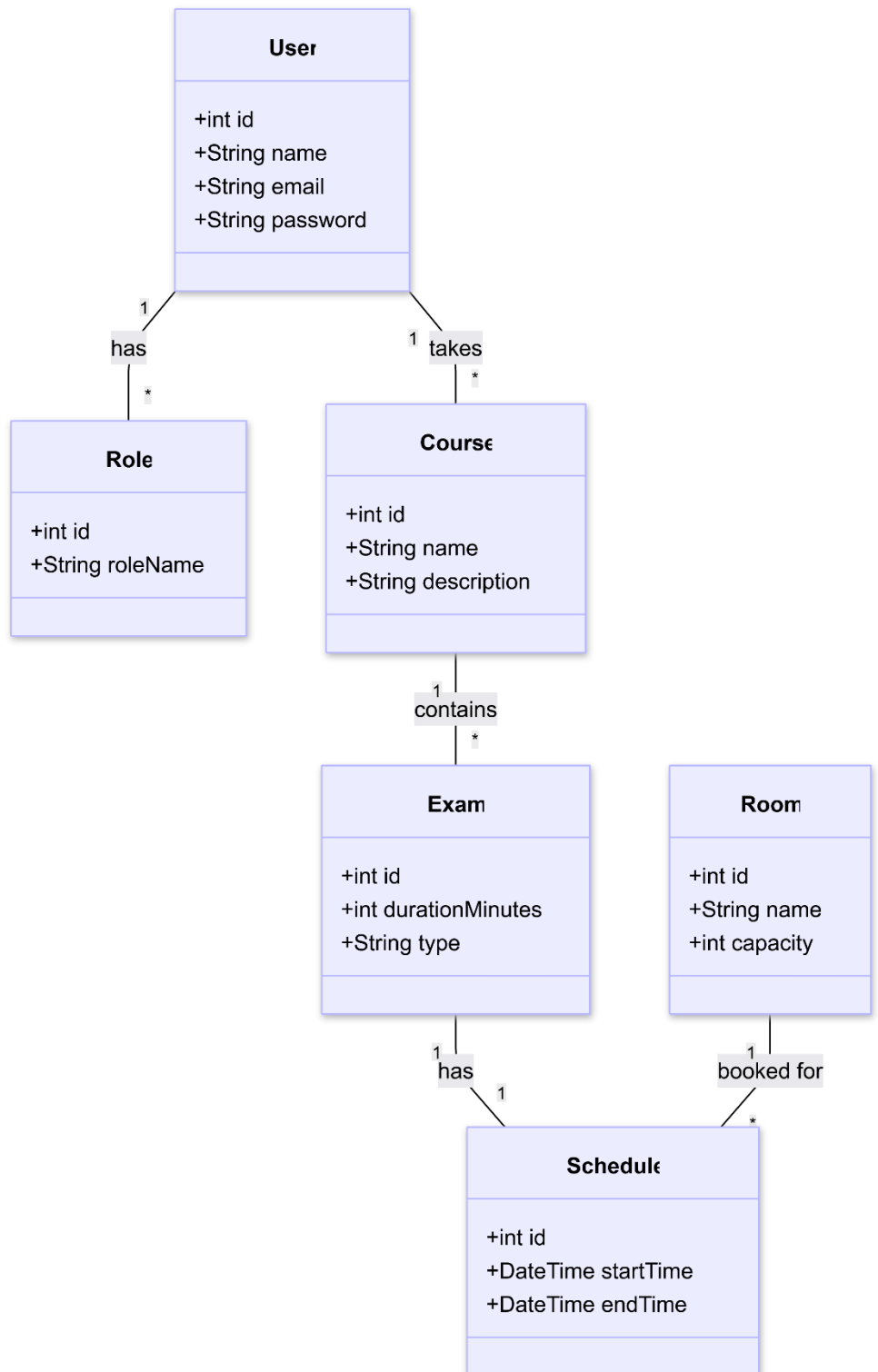
6.1. Diagrama de clase

Această diagramă reprezintă structura statică a sistemului și modul în care datele sunt organizate.

- Entitatea User & Role: Ilustrează un sistem de autorizare bazat pe roluri. Un User poate avea mai multe roluri, permițând flexibilitate (de exemplu, un utilizator poate fi și profesor și administrator).

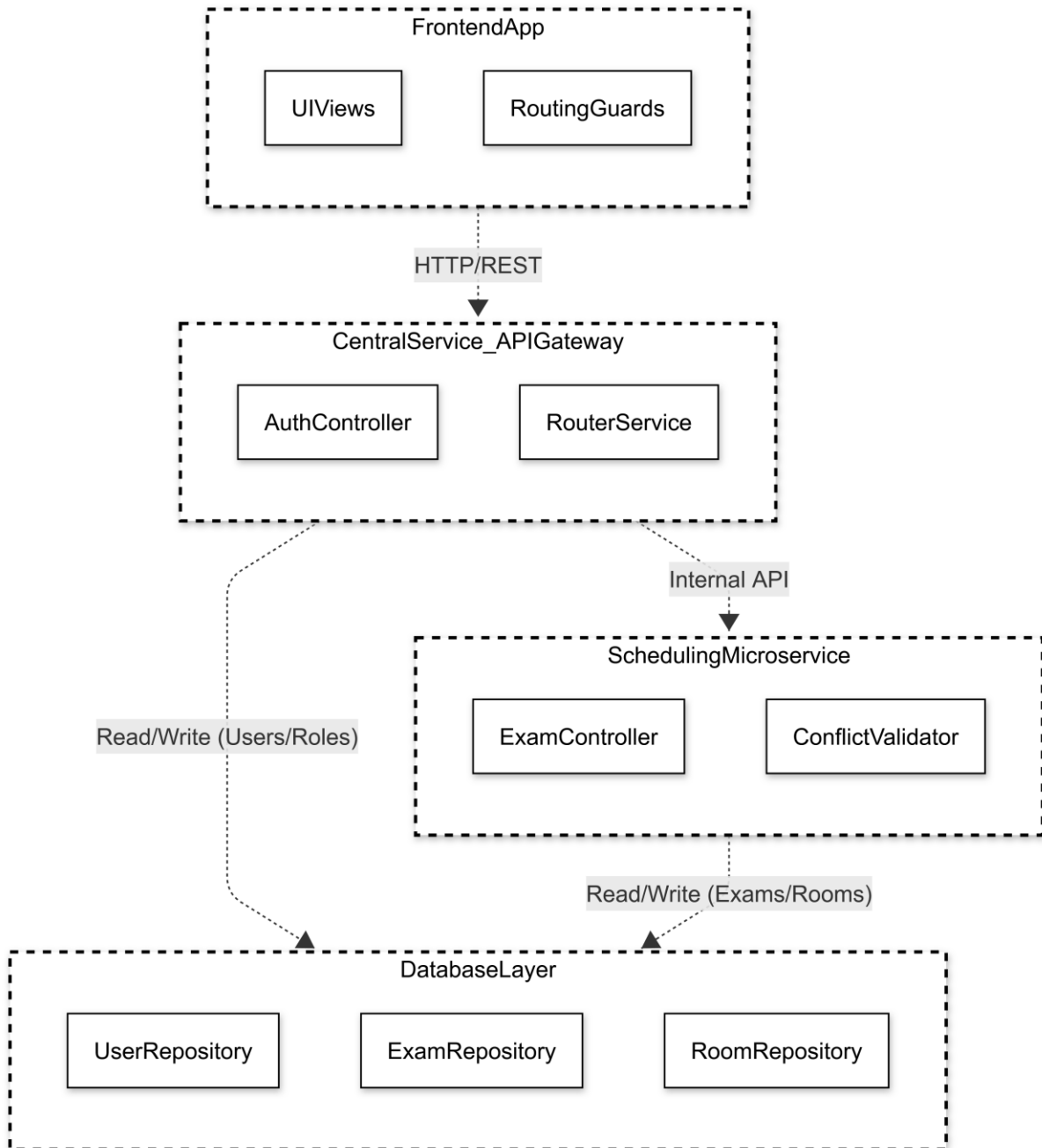
- Course & Exam: Relația de tip 1:* (one-to-many) arată că un curs poate genera mai multe sesiuni de examinare (ex: restante, parțiale), dar fiecare examen este strict legat de un conținut academic specific.

- Schedule & Room: Clasa Schedule face legătura între un Exam, un Room și un interval de timp (startTime, endTime), asigurând astfel unicitatea unei rezervări și evitarea suprapunerilor.



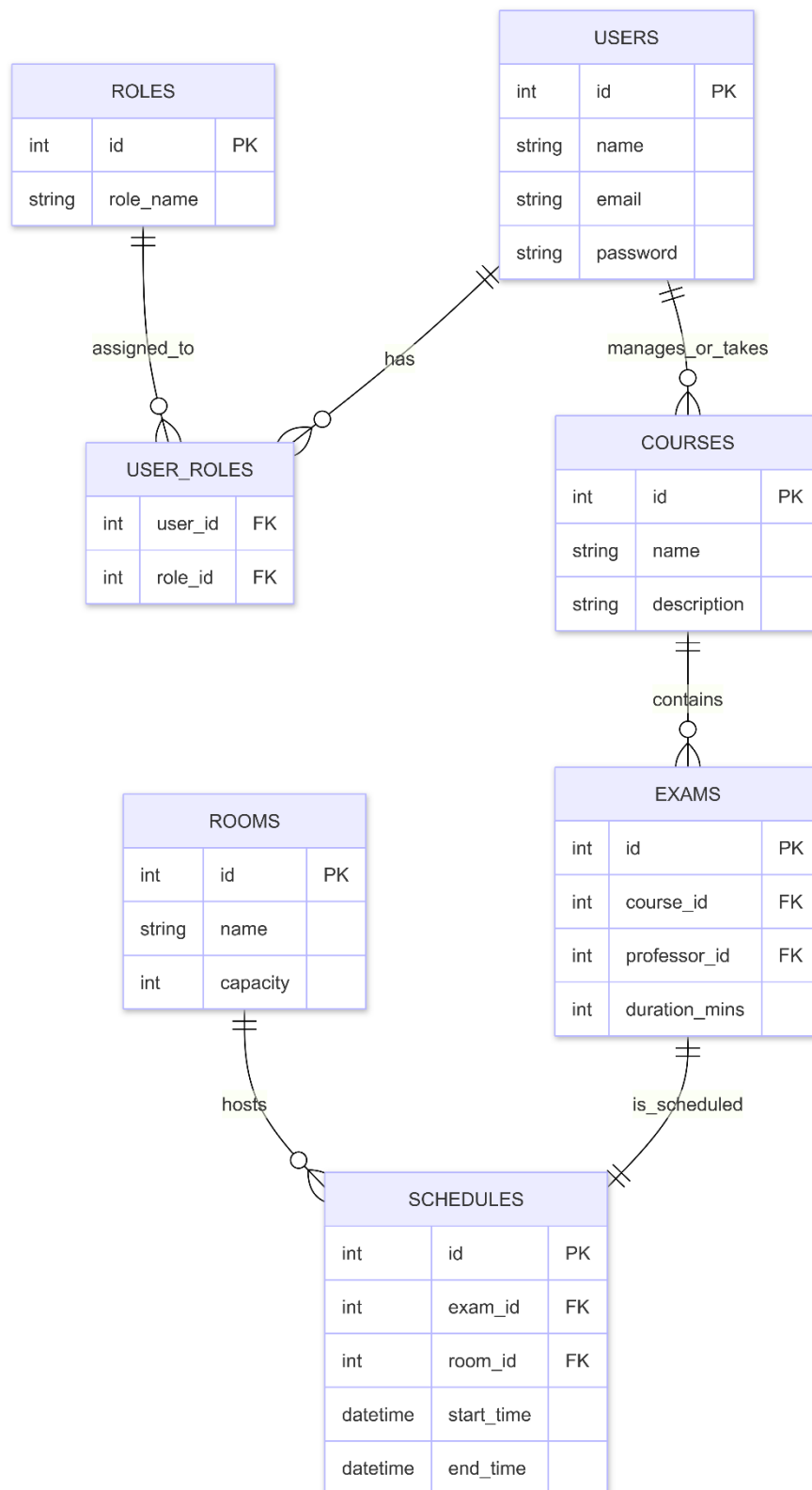
6.2. Diagrama de pachete

Explică organizarea logică a codului și separarea responsabilităților

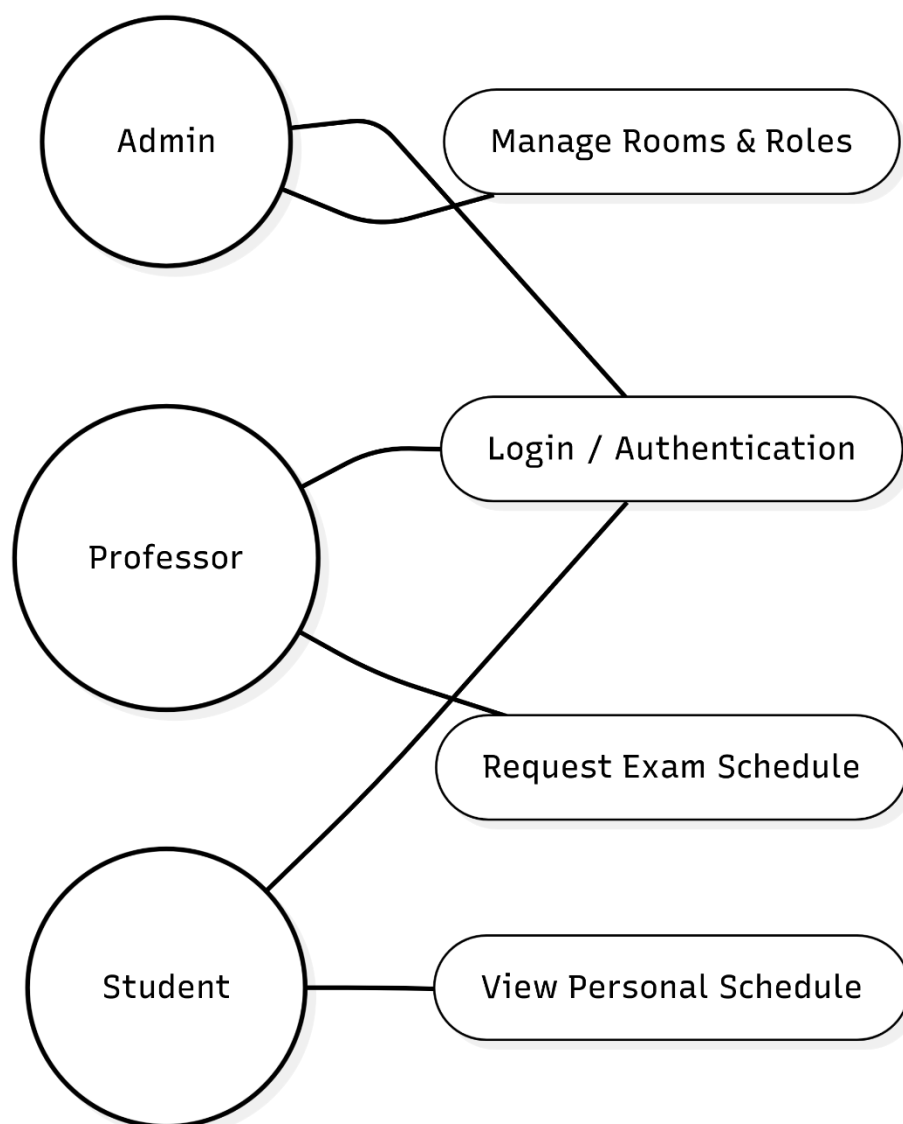


6.3. Diagrama bazei de date

Reprezintă modul în care sunt stocate informațiile în baza de date.



6.4. Diagrama Use-Case



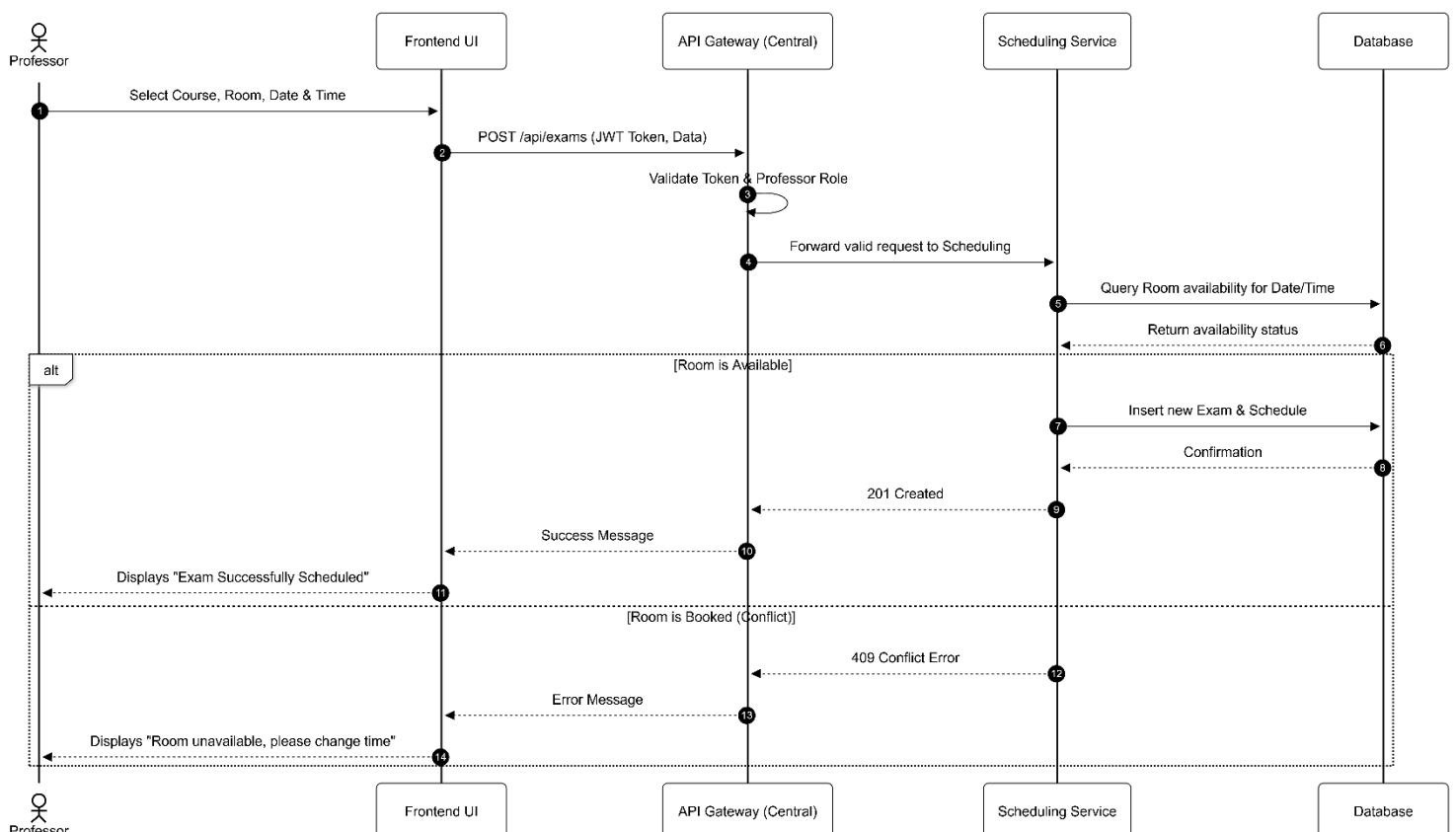
Definirea cazurilor de utilizare:

- **Login / Authentication**
Permite utilizatorilor (Admin, Profesor, Student) să se autentifice în sistem folosind credențiale (email/parolă) pentru a accesa funcționalitățile specifice rolului lor.
- **Manage Rooms & Roles (Admin)**
Adminul poate gestiona sălile (adăugare, editare, ștergere) și rolurile utilizatorilor (atribuire drepturi, modificare roluri).
- **Request Exam Schedule (Professor)**
Profesorul poate trimite o cerere pentru programarea unui examen, specificând detalii precum dată, oră și sală dorită.
- **View Personal Schedule (Student)**
Studentul își poate vedea programul personal de examene (date, ore, săli alocate).

6.5. Diagrama de secvențe

Scenariu: Creare examen

Cererea trece prin API Gateway pentru validarea rolului , ajunge la Scheduling Service pentru verificarea disponibilității sălii și se finalizează cu un răspuns de succes (201 Created) sau eroare (409 Conflict). Această tratare a erorilor asigură cerința funcțională de eliminare a conflictelor de orar.



Scenariu: Autentificare

Arată fluxul de securitate bazat pe JWT. Se observă utilizarea hashing-ului BCrypt pentru compararea parolelor în baza de date, asigurând un nivel înalt de protecție a datelor sensibile.

